

Simulación de monitoreo ambiental

MPI

Universidad Politécnica Salesiana

Integrantes: Jorge Cueva, Karen Quito, Mateo Barzallo

Estación

2

Clase que simula cada una de las estaciones en Cuenca

Contienen:

- Id
- Nombre
- Zona
- Variables
 - Temperatura
 - Humedad
 - CO2

- Generar Mediciones por ciclo
Tomamos la tema de las 3 variables como un ciclo.

- Generar Medicion
Tomamos como medición el calculo o recepción de una variable.

Medición

3

Clase que contiene la información de cada una de las mediciones

Contienen:

- Estación
- Zona
- Variable
- Valor
- Timestamp

Esta clase nos permite gestionar como objeto a cada una de las mediciones generadas por las estaciones.

Alerta Ambiental

4

Clase que permite generar alertas

Contienen:

- Zona
- Variable
- Valor
- Umbral

Clase que se genera solo si la medición excede los umbrales

Clase para contener la lógica de recepción y procesamiento de mediciones

Dividida en

`Procesar_mediciones()` que se encarga de recibir e iniciar el procesamiento de datos

`Realizar_analisis_pesado()` en donde simulamos el procesamiento que se debe realizar con esos datos

`Verificar_alerta()` en donde se revisa si se debe crear o no una alerta en esa medición.

En esta clase vamos a operar las distintas versiones de ejecución, este es nuestro programa

Lo Inicializamos con la cantidad de estaciones que queremos, (4,8,12)

Registra_medición

en donde:



Recibe información

Verifica alertas

Procesa Mediciones

Almacenar Mediciones

Conexión

Red interna

Configuración de DNS

Slaves con conexión ssh
configuración de claves RSA

Carpeta compartida por red
(samba)

Ejecución

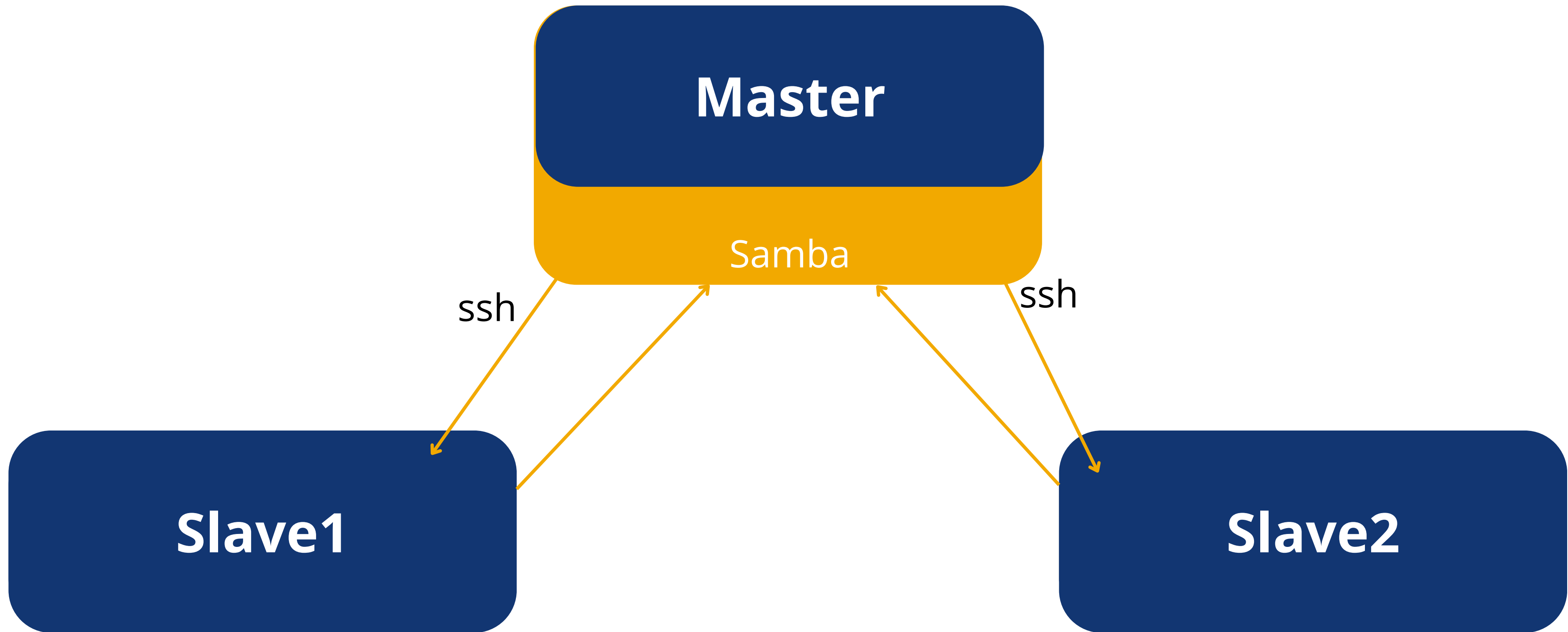
MPI x cantidad de nodos
4,8,12

+ el nodo de la ui
Podemos distribuir los
nodos a gusto

Pruebas y resultados

Ejecutamos la aplicación con
distinta cantidad de nodos
y distribuyendola de formas
distintas entre los slaves

hacemos pruebas de tiempos



Master

Samba

Main.py

Modelos

Core

- El modelo de paso de mensajes de MPI con mpi4py permite escalar una aplicación por encima de la cantidad de núcleos físicos de una sola computadora, dividiendo la carga entre múltiples máquinas.
- Se asimiló la diferencia crítica entre concurrencia por memoria compartida (Locks/Barriers en RAM) y comunicación distribuida (paso de mensajes serializados e independientes con mpi4py).
- MPI permite aumentar el número de procesos dependiendo la cantidad de núcleos disponibles en el cluster.
- Con MPI podemos aumentar el número de procesos ejecutados al trabajar con la cantidad de núcleos disponibles en todos los nodos.